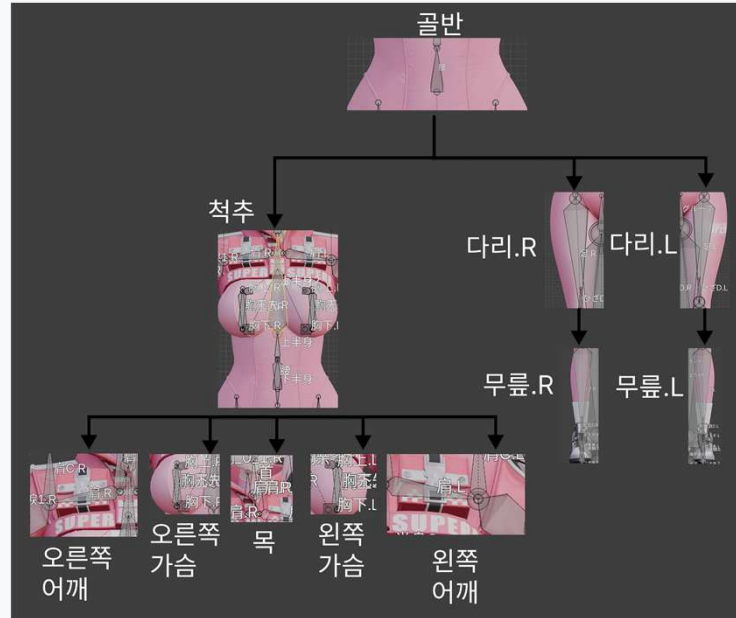
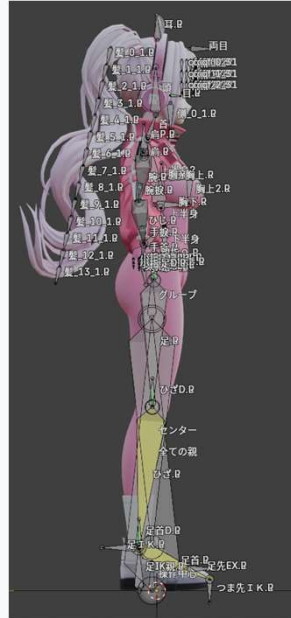
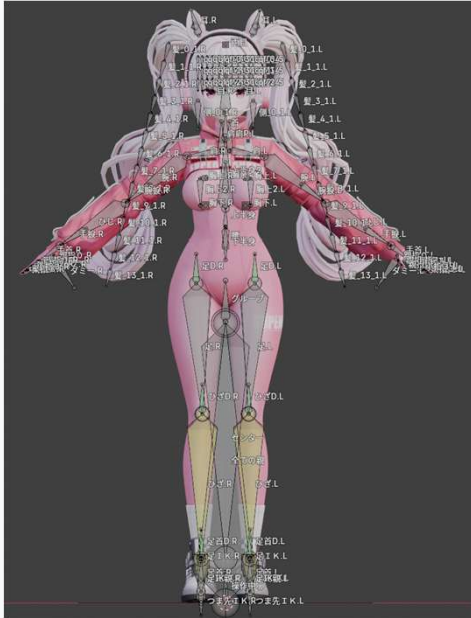


Animation+ (고급 애니메이션)

이창진

Bone Skeleton



```
struct FbxSkeletonNode
{
    std::string name;
    std::wstring nameW;
    int parent = -1;
    std::vector<int> children;
    bool isBone = false;
};
```

<Skeletal Bone 구조>

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
								
[Pelvis]	[spine]	[R.thigh]	[L.thigh]	[Neck]	[R.clavicle]	[L.Clavicle]	[R.calf]	[L.calf]
ParentIndex = -1	ParentIndex = 0	ParentIndex = 0	ParentIndex = 0	ParentIndex = 1	ParentIndex = 1	ParentIndex = 1	ParentIndex = 2	ParentIndex = 2

aiAnimation

캐릭터에 애니메이션이 존재하면 aiAnimation 구조체 내의 **aiNodeAnim**이 존재하게 됩니다. nullptr 체크로 존재 유무를 알아내면 됩니다. 애니메이션 진행 시간에 맞는 **Local Transform**을 알아낼 수 있습니다

aiAnimation: 애니메이션 전체 정보

재생시간(초) = $mDuration / mTicksPerSecond$

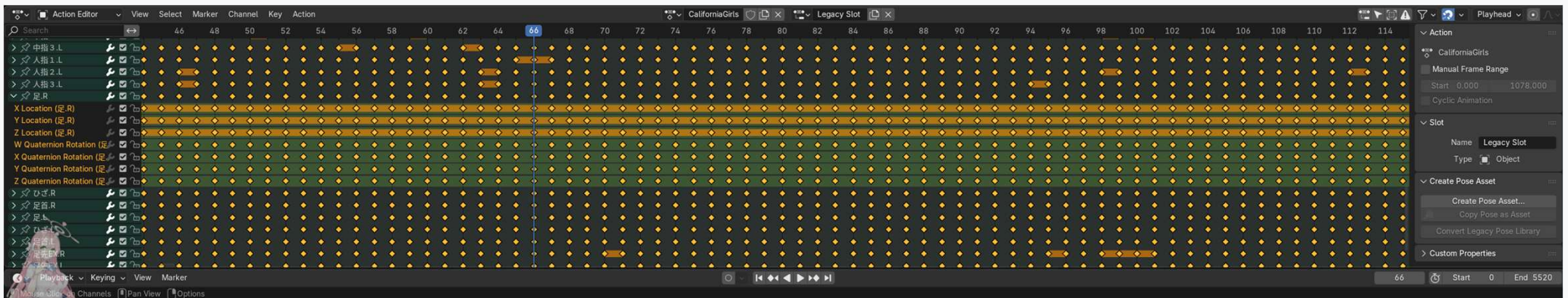
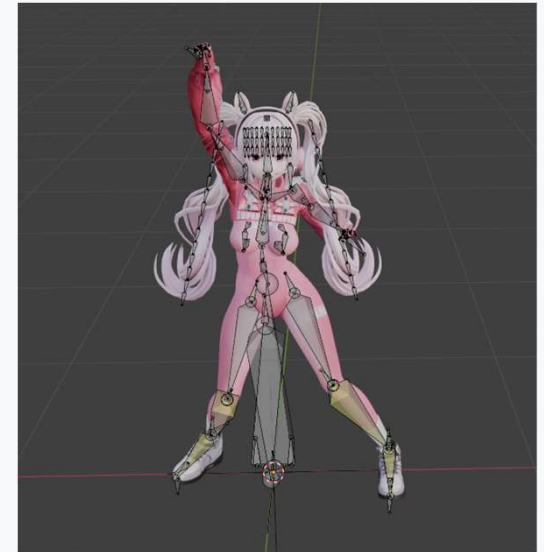
mNumChannels = 움직이는 본인 노드의 트랙 개수

aiNodeAnim: 노드 1개의 트랙

PositionKeys / RotationKeys / ScalingKeys
= 시간별 위치·회전·스케일 키프레임 목록

```
struct aiAnimation {  
    C_STRUCT aiString mName;  
    double mDuration;  
    double mTicksPerSecond;  
    unsigned int mNumChannels;  
    C_STRUCT aiNodeAnim **mChannels;  
    unsigned int mNumMeshChannels;  
    C_STRUCT aiMeshAnim **mMeshChannels;  
    unsigned int mNumMorphMeshChannels;  
    C_STRUCT aiMeshMorphAnim **mMorphMeshChannels;  
};
```

```
struct aiNodeAnim {  
    C_STRUCT aiString mNodeName;  
    unsigned int mNumPositionKeys;  
    C_STRUCT aiVectorKey *mPositionKeys;  
    unsigned int mNumRotationKeys;  
    C_STRUCT aiQuatKey *mRotationKeys;  
    unsigned int mNumScalingKeys;  
    C_STRUCT aiVectorKey *mScalingKeys;  
    C_ENUM aiAnimBehaviour mPreState;  
    C_ENUM aiAnimBehaviour mPostState;  
};
```



Key Interp

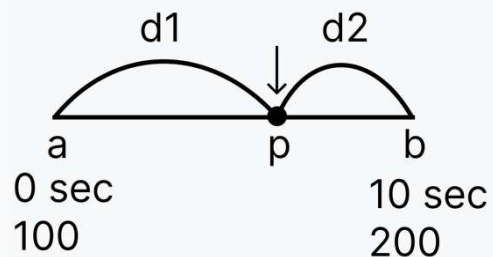
애니메이션 키프레임은 보통 1초에 30개(30 프레임) 정도만 들어있습니다.

그래서 60 프레임로 더 부드럽게 재생을 하려면, 현재 시간 t 가 들어있는 구간의 키1(시간,값) ~ 키2(시간,값) 사이 값을 보간(interpolation) 해서 계산해야 합니다.

Position, Scale은 선형 보간을 사용합니다

선형보간

DirectX::SimpleMath::Vector3::Lerp



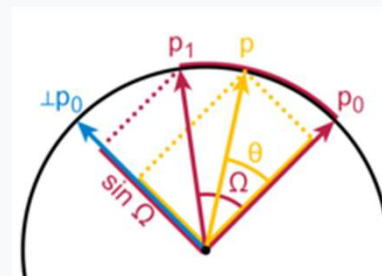
Quaternion은 구면 보간을 사용하는 이유

Quaternion은 단위구(정규화된 공간) 위의 값이어서, **Lerp**는 구 표면이 아니라 내부를 직선으로 섞어 단위가 깨지고 회전이 부자연스러워집니다. **Slerp**는 구 표면을 따라 최단 경로로 보간해서 자연스럽게 일정한 회전이 됩니다

Quaternion은 구면 보간을 사용합니다

구면보간

DirectX::SimpleMath::Quaternion::Slerp



Animation blending

정의

전환 블렌드는 두 포즈를 같은 스켈레톤 공간(동일한 본 순서/리깅)에 맞춰 평가한 뒤 가중치로 섞는 과정입니다

수학적 정의

$$t = \text{elapsed} / \text{blendDuration}$$

$$\text{pose} = \text{Lerp}(\text{poseA}, \text{poseB}, \text{saturate}(t))$$

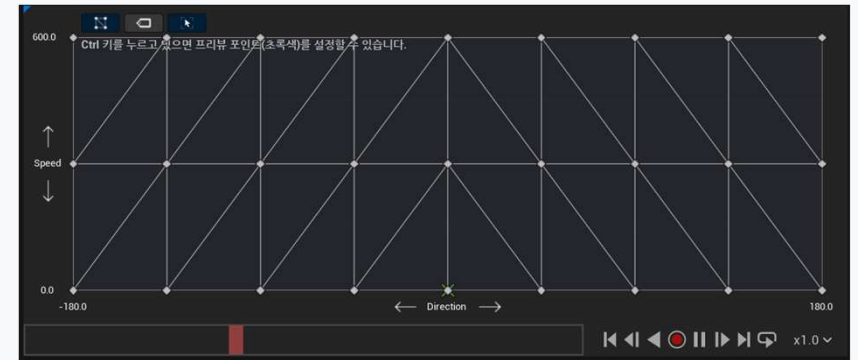
A애니메이션, B 애니메이션이 있을 때, **A는 애니메이션 재생을 유지하면서, B는 처음부터 진행하도록 샘플 시간을 설계해야 합니다**

이를 위해선 전환 시작 시점에 A의 현재 timeA를 고정하거나(또는 계속 진행할 수도 있습니다) B는 시간을 0부터 진행합니다.

$$\text{blendAlpha} = (\text{transitionElapsed} / \text{transitionDuration})$$

$$\text{최종 포즈} = \text{lerp}(\text{poseA}(\text{timeA}), \text{poseB}(\text{timeB}=\text{transitionElapsed}), \text{blendAlpha})$$

전환이 끝나면 state를 B로 바꾸고 timeB를 계속 진행합니다.



Unreal Engine Blend Space



Unity Engine Animation Blending

Additive Animation

정의

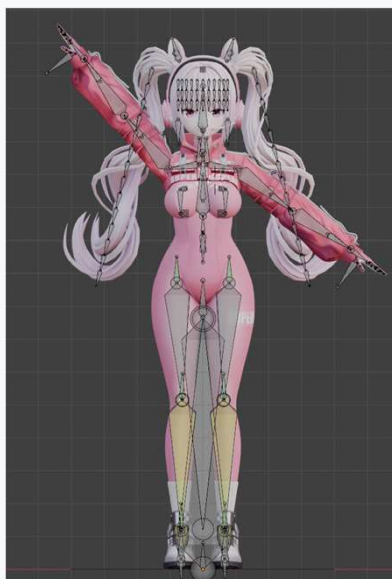
Additive는 **Base 포즈** 대비 **Target 포즈**의 변화량을 계산해 **Base 포즈에 더하는 방식**입니다. 회전은 쿼터니언을 사용하여 상대 회전을 누적(또는 Slerp 기반)하는 방식으로 사용합니다

수학적 정의

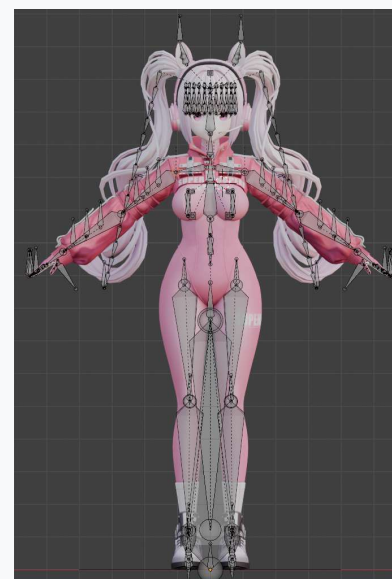
translation: $\text{base} + (\text{add} - \text{ref}) * w$

scale: $\text{base} * \text{lerp}(1, \text{add}/\text{ref}, w)$ 또는 $\text{base} + (\text{add} - \text{ref}) * w$

rotation: $\text{base} * \text{slerp}(\text{identity}, \text{add} * \text{inverse}(\text{ref}), w)$



Target 포즈 본
각 본의
Scale
Rotation
Translation



Base 포즈 본
각 본의
Scale
Rotation
Translation

배기

= 두 포즈의
키프레임 차이 값

Socket

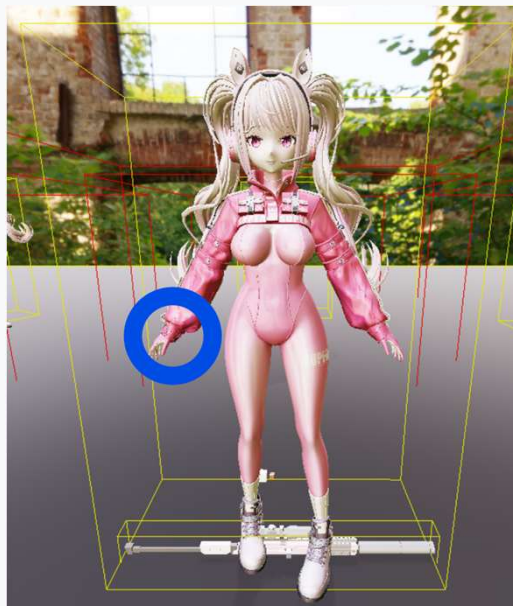
정의

소켓(Socket)은 본에 부착할 수 있는 가상의 연결점입니다. 손에 해당하는 본에 소켓을 만들어 무기를 부착할 수 있습니다.

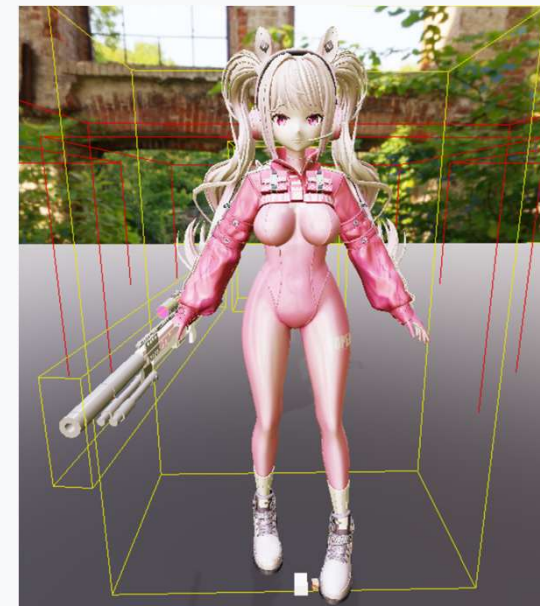
구현

T * R * S 순서로 구성하여 부모 본의 로컬 공간에서 올바른 위치/회전을 만듭니다

캐릭터의 월드에서의 트랜스폼 변화도 반영해야 합니다. 이를위해서 캐릭터 월드 행렬을 곱합니다



소켓위치



무기적용

AnimGraph / Blend Debug			
[x] Enable AdvancedRig			
Rig Initd: true			
- Weapon Socket Transform			
[WeaponPoint] attached to [??R]			
0.0000	0.0000	0.0000	Socket Pos
0.000	0.000	0.000	Socket Rot
1.000	1.000	1.000	Socket Scale

IK

정의

IK(Inverse Kinematics)는 목표 지점에 도달하기 위해 관절 체인의 각도를 계산하는 기술입니다.

쉽게말해서 손을 움직이면 팔이 따라온다를 구현하는 기술입니다

구현

CCD(Cyclic Coordinate Descent) 알고리즘을 사용합니다.
끝에서부터 하나씩, 목표를 향해 조금씩 꺾는 알고리즘 입니다

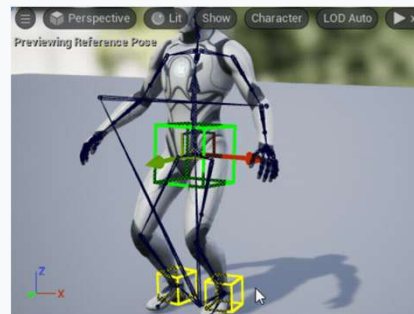
애니메이션 키만으로는 맞추기 힘든 **발 붙이기, 손 뻗기, 지형 대응**
같은 걸 자연스럽게 처리하기 위해서 필요합니다

CCD 알고리즘

1. 가장 끝 관절(End Effector)을 목표 지점을 바라보도록 회전시킵니다.
2. 그 바로 위의 부모 관절로 이동해서, 다시 끝 관절이 목표 지점에 가까워지도록 부모를 회전시킵니다.
3. 이를 루트(Root) 관절까지 반복합니다.
4. 목표에 도달할 때까지 이 과정을 반복합니다.

언리얼 문서 참고

https://dev.epicgames.com/documentation/ko-kr/unreal-engine/ccdik?application_version=4.27



Animation Layers

상하체 분리 재생

Animation Layer는 하체와 상체를 독립적으로 제어할 수 있게 해줍니다. 예를 들어, 걷는 동작(Base)을 하면서 동시에 사격 자세(Upper)를 취할 수 있습니다. 쉽게 말해서 마치 포토샵의 레이어처럼, **신체 부위별로 다른 애니메이션을 덮어씌우는 기술**입니다.

구현

1. 영역 나누기 (Bone Masking)

이름을 보고 상체인지 확인합니다. 뼈 이름(Bone Name)에 특정 단어가 들어가는지 검사합니다. 예를들어 다음과 같은 이름입니다.
"Spine", "Arm", "척추", "상체", "腕(팔)" 이를 upper body라고 해둡니다

2. 강도 조절 (Alpha Blending)

투명도(Alpha)로 얼마나 세게 덮어쓸지 정합니다. 파라미터 값으로 layerAlpha (0.0 ~ 1.0)를 사용합니다. 이는 다음과 같습니다.

0.0: 상체 레이어 무시, 1.0: 상체 레이어 완전 적용, 0.5: 반반 섞임

3. 최종 합성 (Override)

모든 뼈를 검사하면서, 상체인 경우에만 계산된 포즈를 덮어씌웁니다.
하체인 경우: continue로 건너뛰고 localsFinal(Base 동작)이 그대로 남습니다.
상체인 경우: BlendMatricesSRT이 끝난 다음 localsFinal에 상체 동작이 덮어씌워집니다

```
if (d.upper.enabled && (d.upper.animA || d.upper.animB) &&
d.upper.layerAlpha > 0.0001f)
{
    // upper A 평가
    ...
    // upper B 평가인데 필요할 때만 함
    ...
    const float upperBlend = Clamp01(d.upper.blend01);
    const float upperAlpha = Clamp01(d.upper.layerAlpha);

    for (size_t i = 0; i < nodeCount; ++i)
    {
        if (!IsUpperBody(m_NodeNames[i])) continue;

        // upper에 채널이 없는 노드는 건드리지 않음. bind는 유지
        const bool aHas = (i < hasUA.size()) ? (hasUA[i] != 0) : false;
        const bool bHas = upperHasB ? ((i < hasUB.size()) ? (hasUB[i] != 0) :
false) : false;

        if (!aHas && !bHas) continue;

        XMATRIX upperLocal = localsUA[i];
        if (upperHasB)
        {
            if (!aHas) upperLocal = localsUB[i];
            else if (!bHas) upperLocal = localsUA[i];
            else upperLocal = BlendMatricesSRT(localsUA[i], localsUB[i],
upperBlend);
        }

        // base 위에 upper를 upperAlpha만큼 덮는다
        localsFinal[i] = BlendMatricesSRT(localsFinal[i], upperLocal,
upperAlpha);
    }
}
```